

ACP_API_DOCUMENTATION

ACP (AI Coding Protocol) API Documentation

Overview

This document describes the API endpoints and data structures related to ACP (AI Coding Protocol) support in the LLM Verifier project.

Data Structures

ACP Feature Detection Result

```
{
  "tool_use": true,
  "functions": [],
  "code_generation": true,
  "code_completion": true,
  "code_review": true,
  "code_explanation": true,
  "embeddings": false,
  "reranking": false,
  "image_generation": false,
  "audio_generation": false,
  "video_generation": false,
  "mcps": true,
  "lsps": true,
  "acps": true,                // ACP support flag
  "multimodal": false,
  "streaming": true,
  "json_mode": true,
  "structured_output": true,
  "reasoning": false,
  "function_calling": true,
  "parallel_tool_use": false,
  "max_parallel_calls": 0,
  "modalities": ["text"],
  "batch_processing": false,
```

```
"supports_brotli": false
}
```

ACP Configuration

```
{
  "enabled": true,
  "protocol_version": "2.0",
  "max_tool_calls": 10,
  "context_window_size": 128000,
  "supports_code_actions": true,
  "supports_diagnostics": true,
  "supports_completion": true,
  "supported_methods": [
    "textDocument/completion",
    "textDocument/hover",
    "textDocument/definition",
    "textDocument/references",
    "workspace/symbol"
  ]
}
```

API Endpoints

1. Verify ACP Support

Endpoint: POST /api/verify/acp

Description: Verify if a specific model supports ACP capabilities.

Request Body:

```
{
  "model_name": "gpt-4",
  "provider": "openai",
  "endpoint": "https://api.openai.com/v1",
  "api_key": "sk-...",
  "test_scenarios": [
    {
      "name": "jsonrpc_compliance",
      "enabled": true
    },
    {
      "name": "tool_calling",
      "enabled": true
    },
    {
      "name": "context_management",
      "enabled": true
    },
    {

```

```

        "name": "code_assistance",
        "enabled": true
    },
    {
        "name": "error_detection",
        "enabled": true
    }
],
"timeout": 30
}

```

Response:

```

{
  "success": true,
  "model_name": "gpt-4",
  "acp_support": {
    "supported": true,
    "confidence": 0.85,
    "capabilities": {
      "jsonrpc_compliance": {
        "supported": true,
        "score": 0.9,
        "details":
          "Model understands JSON-RPC format and responds
          appropriately"
      },
      "tool_calling": {
        "supported": true,
        "score": 0.8,
        "details": "Model can handle tool calling requests"
      },
      "context_management": {
        "supported": true,
        "score": 0.85,
        "details": "Model maintains context across conversation
          turns"
      },
      "code_assistance": {
        "supported": true,
        "score": 0.9,
        "details": "Model provides helpful code generation and
          completion"
      },
      "error_detection": {
        "supported": true,
        "score": 0.75,
        "details": "Model can identify and explain code errors"
      }
    }
  },
  "overall_score": 0.84
},
"execution_time": "2.3s",

```

```
"timestamp": "2025-12-27T21:29:18.254433+03:00"
}
```

Error Response:

```
{
  "success": false,
  "error": {
    "code": "ACP_TEST_FAILED",
    "message": "ACP capability test failed",
    "details": "Model did not respond to JSON-RPC requests",
    "retryable": true
  }
}
```

2. Get ACP Configuration

Endpoint: GET /api/config/acp/{provider}

Description: Get ACP configuration for a specific provider.

Path Parameters:

- provider (string, required): Provider name (e.g., "openai", "anthropic", "deepseek")

Response:

```
{
  "provider": "openai",
  "acp_config": {
    "enabled": true,
    "protocol_version": "2.0",
    "max_tool_calls": 10,
    "context_window_size": 128000,
    "supports_code_actions": true,
    "supports_diagnostics": true,
    "supports_completion": true,
    "supported_methods": [
      "textDocument/completion",
      "textDocument/hover",
      "textDocument/definition"
    ],
    "default_timeout": 30,
    "retry_config": {
      "max_retries": 3,
      "backoff_factor": 2.0,
      "initial_delay": 1000
    }
  }
}
```

3. Update ACP Configuration

Endpoint: PUT /api/config/acp/{provider}

Description: Update ACP configuration for a specific provider.

Path Parameters:

- provider (string, required): Provider name

Request Body:

```
{
  "acp_config": {
    "enabled": true,
    "protocol_version": "2.0",
    "max_tool_calls": 15,
    "context_window_size": 200000,
    "supports_code_actions": true,
    "supports_diagnostics": true,
    "supports_completion": true,
    "supported_methods": [
      "textDocument/completion",
      "textDocument/hover",
      "textDocument/definition",
      "textDocument/references"
    ],
    "default_timeout": 45,
    "retry_config": {
      "max_retries": 5,
      "backoff_factor": 1.5,
      "initial_delay": 500
    }
  }
}
```

Response:

```
{
  "success": true,
  "message": "ACP configuration updated successfully",
  "provider": "openai",
  "updated_config": {
    "enabled": true,
    "protocol_version": "2.0",
    "max_tool_calls": 15,
    "context_window_size": 200000,
    "supports_code_actions": true,
    "supports_diagnostics": true,
    "supports_completion": true,
    "supported_methods": [
      "textDocument/completion",
      "textDocument/hover",
      "textDocument/definition",

```

```

        "textDocument/references"
    ],
    "default_timeout": 45,
    "retry_config": {
        "max_retries": 5,
        "backoff_factor": 1.5,
        "initial_delay": 500
    }
}
}

```

4. List Models with ACP Support

Endpoint: GET /api/models/acp

Description: List all models that support ACP capabilities.

Query Parameters:

- provider (string, optional): Filter by provider
- min_score (number, optional): Minimum ACP score threshold
- limit (integer, optional): Maximum number of results (default: 100)
- offset (integer, optional): Pagination offset

Response:

```

{
  "success": true,
  "models": [
    {
      "model_id": "gpt-4",
      "provider": "openai",
      "acp_score": 0.85,
      "acp_capabilities": {
        "jsonrpc_compliance": true,
        "tool_calling": true,
        "context_management": true,
        "code_assistance": true,
        "error_detection": true
      },
      "last_verified": "2025-12-27T21:29:18.254433+03:00"
    },
    {
      "model_id": "claude-3-opus",
      "provider": "anthropic",
      "acp_score": 0.82,
      "acp_capabilities": {
        "jsonrpc_compliance": true,
        "tool_calling": true,
        "context_management": true,
        "code_assistance": true,
        "error_detection": false
      },
    },
  ]
}

```

```

        "last_verified": "2025-12-27T21:29:18.254433+03:00"
    }
],
"pagination": {
    "total": 25,
    "limit": 100,
    "offset": 0,
    "has_more": false
}
}

```

5. Get ACP Test Results

Endpoint: GET /api/results/acp/{model_id}

Description: Get detailed ACP test results for a specific model.

Path Parameters:

- model_id (string, required): Model identifier

Response:

```

{
  "success": true,
  "model_id": "gpt-4",
  "acp_results": {
    "supported": true,
    "score": 0.85,
    "last_tested": "2025-12-27T21:29:18.254433+03:00",
    "test_details": {
      "jsonrpc_compliance": {
        "test": "JSON-RPC protocol comprehension",
        "passed": true,
        "score": 0.9,
        "response_time": "0.5s",
        "details": "Model correctly interpreted JSON-RPC request and provided valid response"
      },
      "tool_calling": {
        "test": "Tool calling capability",
        "passed": true,
        "score": 0.8,
        "response_time": "0.6s",
        "details": "Model demonstrated understanding of tool calling concepts"
      },
      "context_management": {
        "test": "Context management",
        "passed": true,
        "score": 0.85,
        "response_time": "0.7s",

```

```

        "details": "Model maintained context across conversation turns"
    },
    "code_assistance": {
        "test": "Code assistance",
        "passed": true,
        "score": 0.9,
        "response_time": "0.8s",
        "details": "Model provided helpful code generation with proper structure"
    },
    "error_detection": {
        "test": "Error detection",
        "passed": true,
        "score": 0.75,
        "response_time": "0.4s",
        "details": "Model identified code errors and suggested improvements"
    }
},
"overall_assessment": "Model demonstrates strong ACP compatibility with excellent JSON-RPC support and code assistance capabilities."
}
}

```

6. Run ACP Challenge

Endpoint: POST /api/challenges/acp

Description: Run a comprehensive ACP challenge test suite.

Request Body:

```

{
  "models": ["gpt-4", "claude-3-opus", "deepseek-chat"],
  "challenge_config": {
    "timeout": 60,
    "max_attempts": 3,
    "scenarios": [
      {
        "name": "complete_workflow",
        "description": "Complete ACP workflow simulation",
        "steps": [
          "jsonrpc_handshake",
          "tool_discovery",
          "code_completion",
          "error_diagnostics",
          "context_switching"
        ]
      }
    ]
  },
  "evaluation_criteria": {

```

```

        "min_score": 0.7,
        "required_capabilities": ["jsonrpc", "tool_calling",
                                   "code_assistance"]
    }
}
}

```

Response:

```

{
  "success": true,
  "challenge_id": "acp-challenge-001",
  "results": [
    {
      "model_id": "gpt-4",
      "overall_score": 0.88,
      "passed": true,
      "scenario_results": [
        {
          "name": "complete_workflow",
          "score": 0.88,
          "passed": true,
          "step_results": {
            "jsonrpc_handshake": {"passed": true, "score": 0.95},
            "tool_discovery": {"passed": true, "score": 0.85},
            "code_completion": {"passed": true, "score": 0.9},
            "error_diagnostics": {"passed": true, "score": 0.8},
            "context_switching": {"passed": true, "score": 0.9}
          }
        }
      ],
      "execution_time": "45.2s"
    }
  ],
  "summary": {
    "total_models": 3,
    "passed_models": 2,
    "failed_models": 1,
    "average_score": 0.82
  }
}

```

WebSocket Real-time Updates

ACP Test Progress

Endpoint: WS /ws/acp/progress/{test_id}

Description: Subscribe to real-time ACP test progress updates.

Message Format:

```
{
  "type": "acp_test_progress",
  "test_id": "acp-test-001",
  "model_id": "gpt-4",
  "progress": {
    "current_test": "tool_calling",
    "completed_tests": 2,
    "total_tests": 5,
    "percentage": 40
  },
  "status": "in_progress",
  "timestamp": "2025-12-27T21:29:18.254433+03:00"
}
```

Error Codes

ACP-specific Error Codes

Code	Description	Retryable
ACP_TEST_FAILED	ACP capability test failed	Yes
ACP_PROTOCOL_ERROR	JSON-RPC protocol error	No
ACP_TIMEOUT	ACP test timed out	Yes
ACP_INVALID_INPUT	Invalid ACP test input	No
ACP_PROVIDER_ERROR	Provider-specific ACP error	Yes
ACP_UNSUPPORTED	ACP not supported by provider	No

General Error Response Format

```
{
  "success": false,
  "error": {
    "code": "ACP_TEST_FAILED",
    "message": "ACP capability test failed",
    "details": "Model did not respond to JSON-RPC requests",
    "retryable": true,
    "suggestions": [
      "Check model availability",
      "Verify API key validity",
      "Try again with longer timeout"
    ]
  }
}
```

Rate Limiting

ACP-specific Rate Limits

- **ACP Verification:** 10 requests per minute per API key
- **ACP Configuration:** 100 requests per hour per API key
- **ACP Results:** 1000 requests per hour per API key

Rate Limit Response

```
{
  "success": false,
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Rate limit exceeded for ACP verification",
    "details": "Limit: 10 requests per minute",
    "retry_after": 30
  }
}
```

Authentication

All ACP API endpoints require authentication using API keys in the header:

Authorization: Bearer YOUR_API_KEY

SDK Examples

Go SDK

```
import "github.com/llmverifier/sdk-go"

client := llmverifier.NewClient("YOUR_API_KEY")

// Verify ACP support
result, err := client.VerifyACP("gpt-4", "openai")
if err != nil {
    log.Fatal(err)
}

fmt.Printf("ACP Support: %t\n", result.Supported)
fmt.Printf("Overall Score: %.2f\n", result.Score)
```

Python SDK

```
import llmverifier

client = llmverifier.Client("YOUR_API_KEY")
```

```
# Verify ACP support
result = client.verify_acp("gpt-4", "openai")
print(f"ACP Support: {result.supported}")
print(f"Overall Score: {result.score:.2f}")
```

JavaScript SDK

```
const LLMVerifier = require('llmverifier');

const client = new LLMVerifier.Client('YOUR_API_KEY');

// Verify ACP support
const result = await client.verifyACP('gpt-4', 'openai');
console.log(`ACP Support: ${result.supported}`);
console.log(`Overall Score: ${result.score.toFixed(2)}`);
```

Changelog

Version 1.0.0

- Initial ACP API implementation
- Basic ACP capability detection
- Provider configuration support
- Real-time progress updates

Version 1.1.0

- Added comprehensive challenge framework
- Enhanced error handling and validation
- Performance optimizations
- WebSocket support for real-time updates

For additional support, please refer to the main API documentation or contact the development team.